LECTURE 1 · WEB TECHNOLOGIES

Introduction to the Internet & HTTP

Md Istakiak Adnan Palash
Lecturer, Dept. of ICT
Bangladesh University of Professionals (BUP)

TODAY'S ROADMAP

What We'll Cover

Birth of the Internet

From ARPANET to a global network

Core Protocols

TCP/IP, DNS — the plumbing of the web

HTTP in Action

The request–response conversation

URLs Decoded

Every part of a web address explained

Request & Response

Anatomy of every HTTP message

HTTP Methods

GET, POST, PUT, DELETE and when to use each

A LITTLE HISTORY

The Birth of the Internet

The Internet traces its roots to a project of the U.S. Department of Defense's Advanced Research Projects Agency (ARPA).

1969
ARPANET launches
Just 4 computers, at 4 sites the very first network.

1972
Email arrives
Electronic mail goes live on ARPANET and quickly becomes hugely popular.

1970s–80s
Universities join in
Local networks and organizations gradually adopt and extend the Internet.

Today
A global network
Billions of computers communicate over TCP/IP on one open, global network.

THE BIG PICTURE

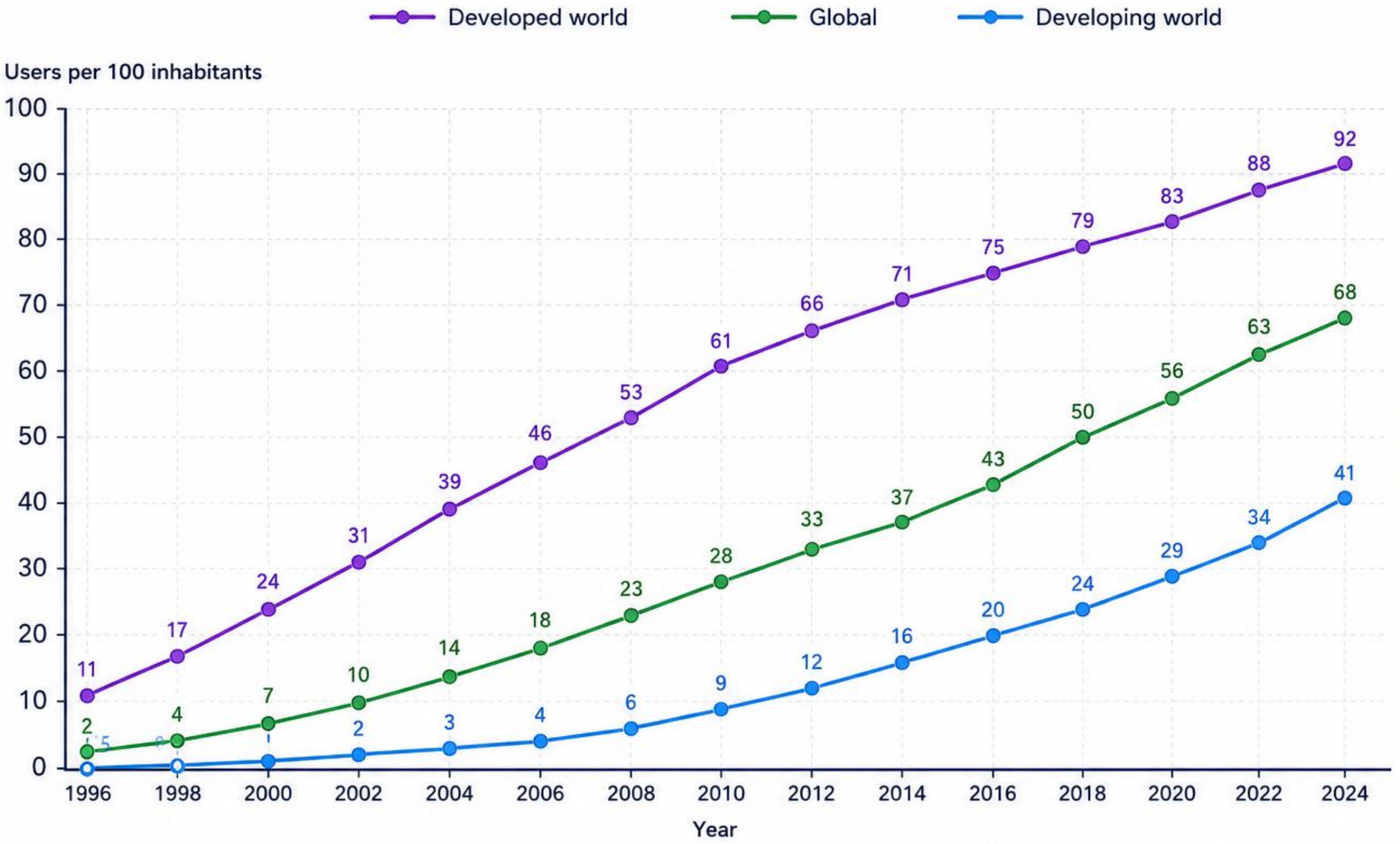
What Is the Internet?

The collection of computers that can communicate with one another using **TCP/IP** over an open, global communications network.

4	1972	Billions
computers connected in 1969	the year email was born	of devices online today

Internet Users Per 100 Inhabitants

Global and regional trends from 1996 to 2024



Latest (2024)

Developed world
92
per 100 inhabitants

Global
68
per 100 inhabitants

Developing world
41
per 100 inhabitants

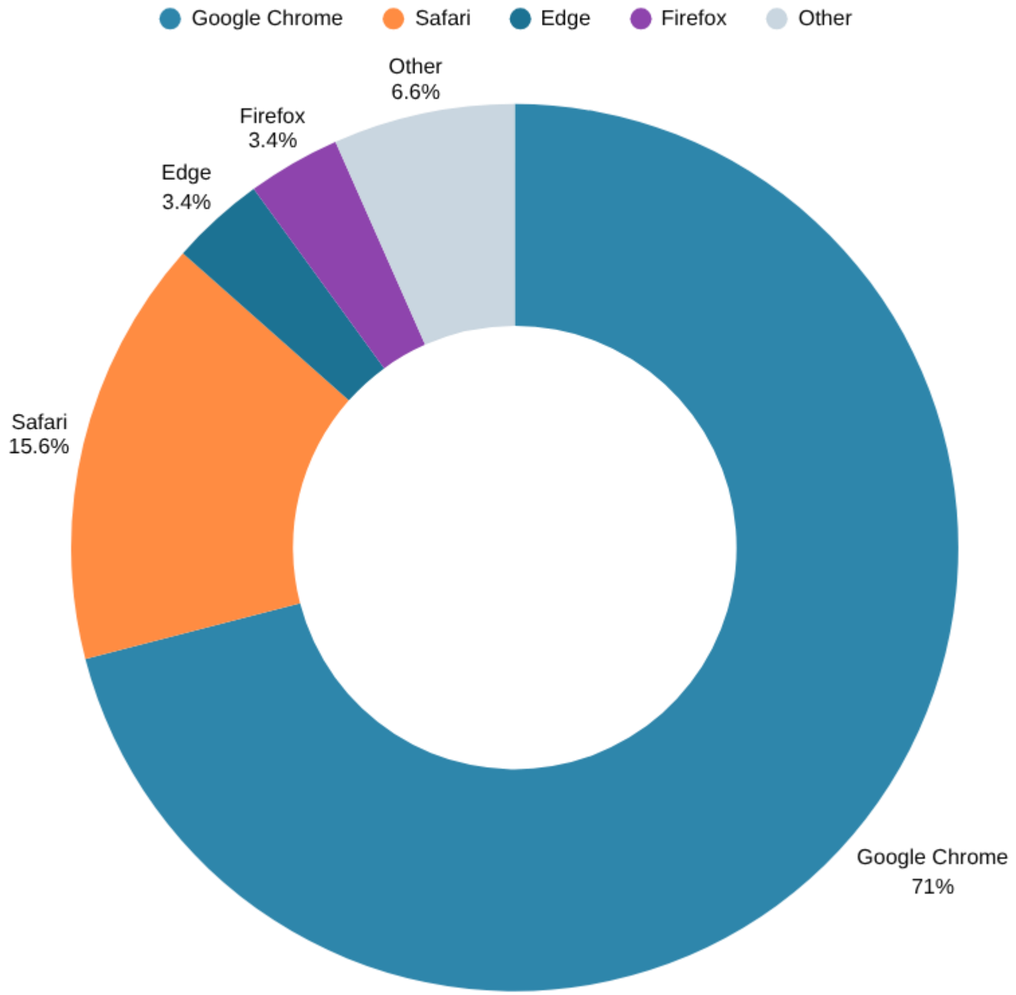
Source: International Telecommunication Union (ITU)
Database: ITU World Telecommunication / ICT Indicators

The developed world includes Europe, North America, Australia & New Zealand and parts of Asia.
The developing world includes all other regions.

WHO'S BROWSING WITH WHAT

Usage Share of All Browsers

StatCounter Global Stats, June 2026 (Worldwide Browser Market Share)



Google Chrome leads by far

Nearly two-thirds of all web traffic in 2026 came through Chrome.

Safari holds strong #2

Driven largely by iPhone and Mac users worldwide.

Edge & Firefox are close

Both sit in a similar single-digit range.

FOUNDATIONS

Web Essentials

In order to understand the web, we need to know:



Basic Internet Protocols

The rules computers follow to exchange data TCP/IP, DNS, and others.



World Wide Web (HTTP)

The protocol that lets browsers and servers talk to each other.



Web Clients

The browser (or app) that requests information on your behalf.



Web Servers

The machine that stores and delivers the content you asked for.

OSI Model

	Layer	Protocol Data Unit (PDU)	Function
Host layers	7 Application	(No specific PDU)	High-level protocols such as for resource sharing or remote file access, e.g. HTTP.
	6 Presentation	Data	Translation of data between a networking service and an application; including character encoding, data compression and encryption/decryption
	5 Session	Data	Managing communication sessions, i.e., continuous exchange of information in the form of multiple back-and-forth transmissions between two nodes
	4 Transport	Segment	Reliable transmission of data segments between points on a network, including segmentation, acknowledgement and multiplexing
Media layers	3 Network	Packet, Datagram	Structuring and managing a multi-node network, including addressing, routing and traffic control
	2 Data Link	Frame	Transmission of data frames between two nodes connected by a physical layer
	1 Physical	101010 Bit, Symbol	Transmission and reception of raw bit streams over a physical medium

BASIC INTERNET PROTOCOLS

TCP/IP: The Language of the Internet

- A protocol is a detailed specification of how communication between two computers is carried out.
- TCP and IP are two different protocols almost always used together, powering email, web browsing, file downloads, and remote databases.



Internet Protocol (IP)

Moves data from a source computer to a destination computer. It packages data, along with source & destination addresses, into a packet.

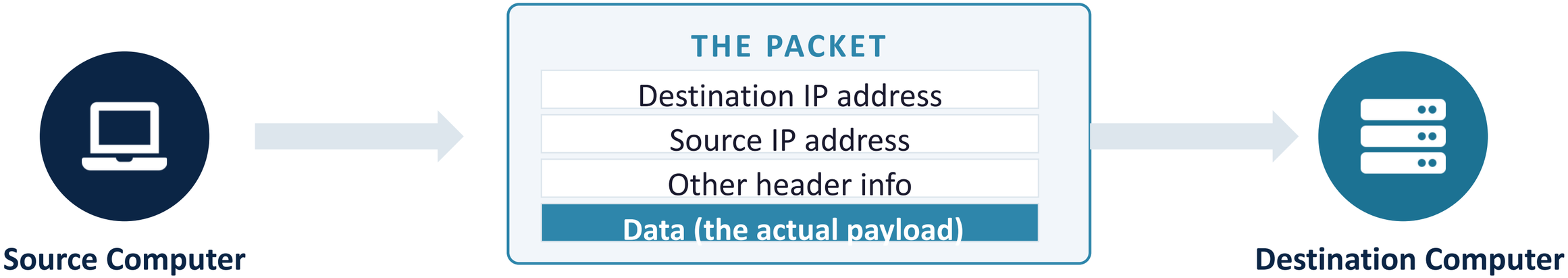


Transmission Control Protocol (TCP)

Adds reliability and the concept of a port letting one machine run many different applications at once.

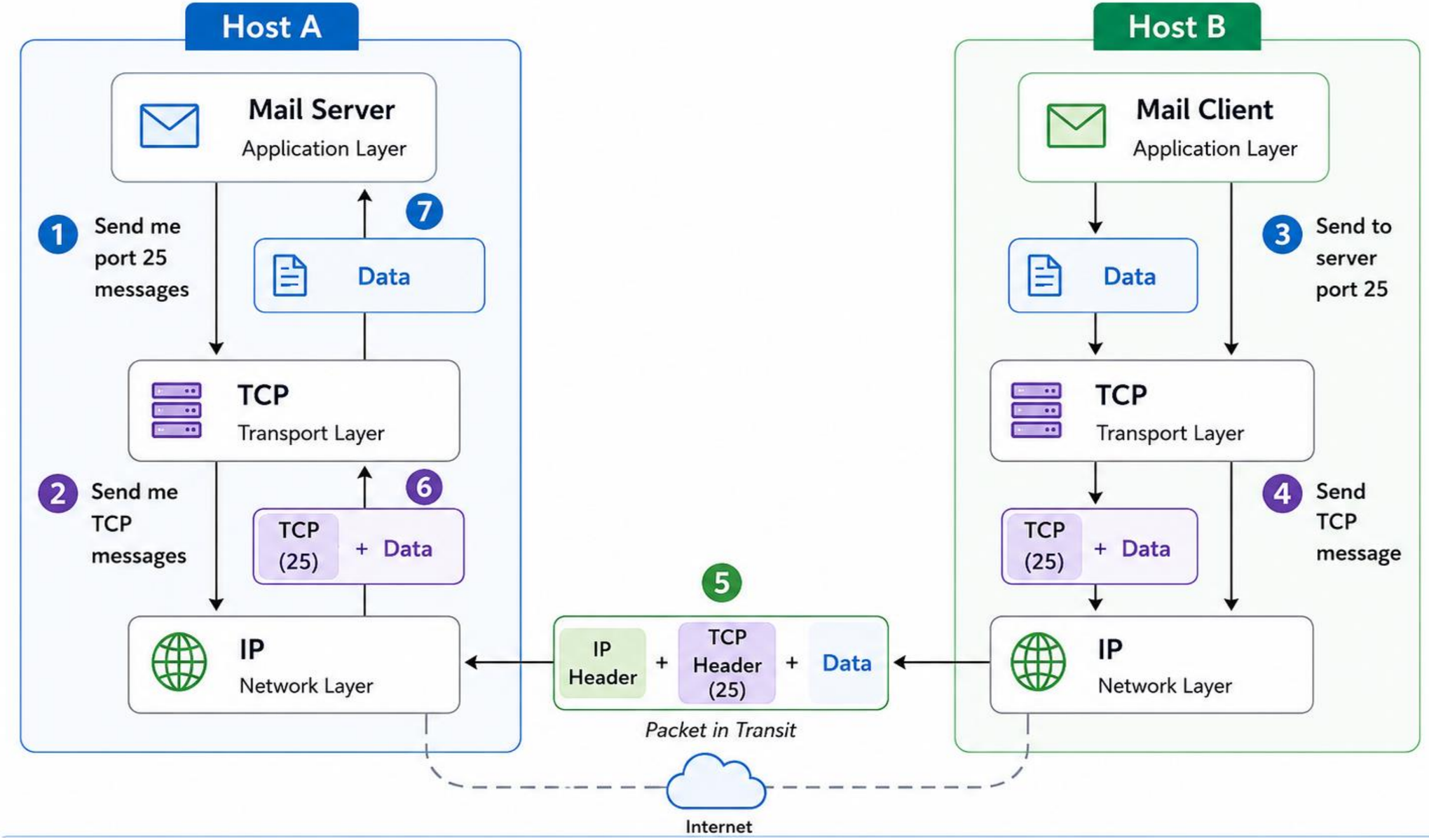
Internet Protocol (IP)

I**P's job:** get a packet of data from a source computer to a destination computer, anywhere on the network.



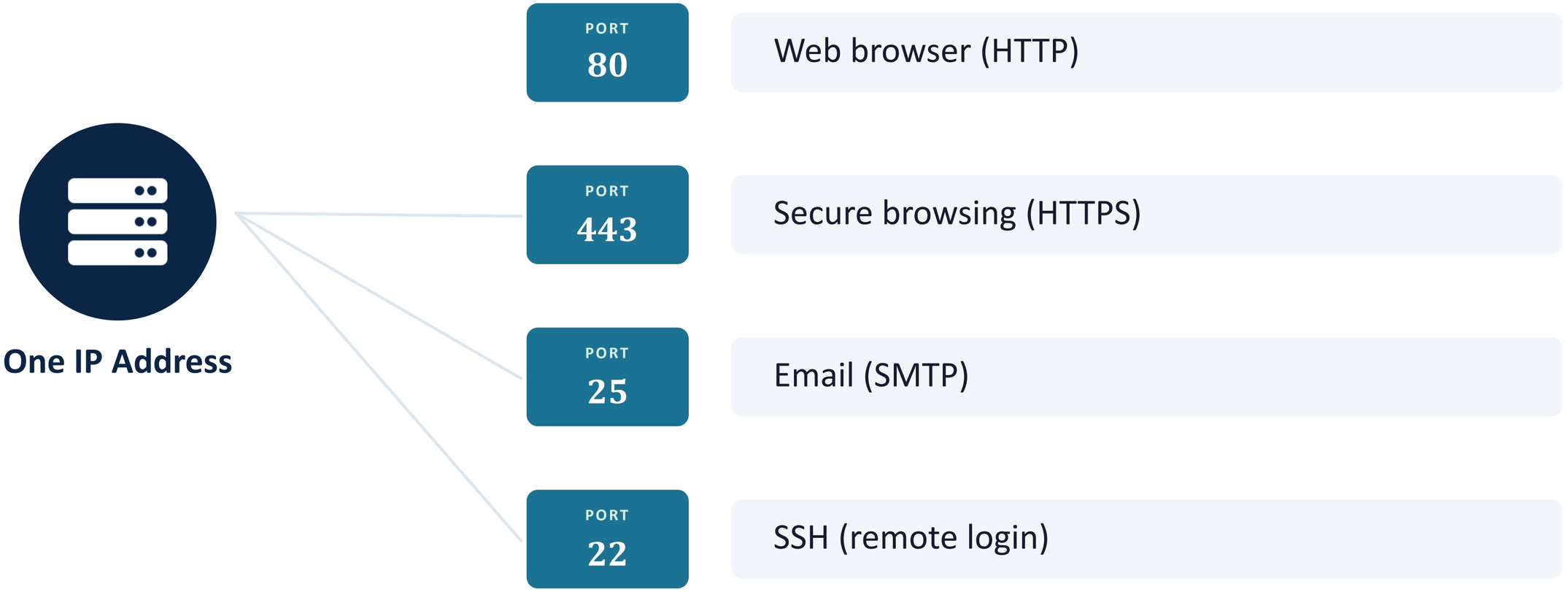
The IP software running on the source creates this packet, a sequence of bits carrying the data plus routing information and sends it towards its destination.

TCP & Ports: One Address, Many Apps



TCP & Ports: One Address, Many Apps

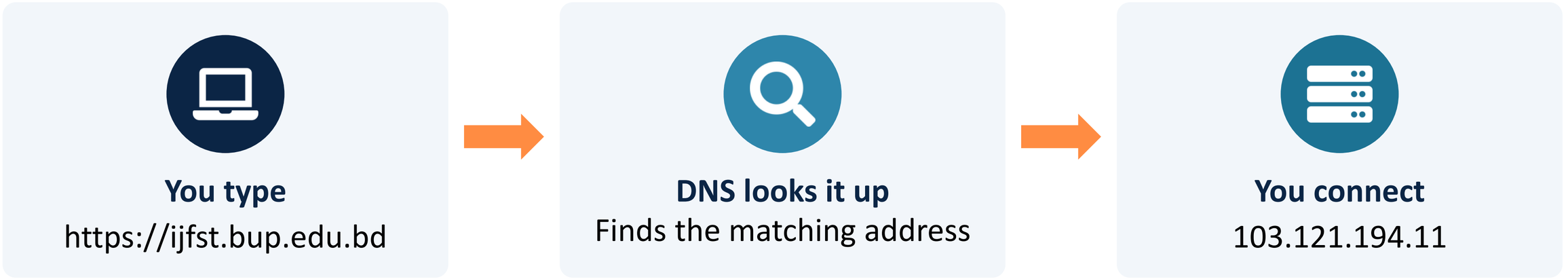
- **TCP** adds multiplexing and guaranteed delivery on top of IP. The **port** concept lets one machine run many programs over the same IP address at once.



Some real-time programs (games, streaming media) use the simpler UDP protocol instead of TCP.

DNS: Turning Names Into Addresses

The Domain Name System is a set of servers that map human-friendly names to numeric IP addresses.



Many systems also keep a local cache called a hosts file:

Windows	`C:\Windows\system32\drivers\etc\hosts`
Mac	`/private/etc/hosts`
Linux	`/etc/hosts`

Hyper Text Transfer Protocol (HTTP)

HTTP is a detailed specification of how web clients and servers should communicate a **request–response** conversation.



HTTP is: connectionless (no lasting link between requests), **media-independent** (can carry any kind of data), and **stateless** (each request stands alone).

PUTTING IT TOGETHER

What Happens When You Type google.com?

Press Enter, and in a fraction of a second, five things happen in sequence:

1



Compose the message

The browser builds a message that conforms to the HTTP protocol.

2



Resolve the name

DNS is used to obtain the IP address for www.google.com.

3



Connect

A TCP connection is created with the machine at that IP address.

4



Send the request

The HTTP message is sent over the TCP connection.

5



Receive & display

A response comes back and is displayed in the browser window.

ADDRESSES ON THE WEB

The Request URI

A URI is an identifier associated with a particular resource like a web page or image on the World Wide Web.

```
http://www.example.org
```



Scheme

The part before the colon (:). Web addresses mostly use the http scheme.



Everything else

Depends on the scheme for http, this points to a location (a resource's address).

Not every scheme points to a location. URN (Uniform Resource Name) identifies a resource by *name* instead — for example:
`urn:ISBN:0-1404-4417-3`

Uniform Resource Locator (URL)

A URL is an identifier for the location of a document on a web site.

```
http:// www.aw-bc.com /info/regesstepp/index.html
```

protocol | host | path

Upon entering this URL, the browser would:

- Ask DNS for the IP address of www.aw-bc.com
- Connect to that IP address at port 80
- Ask the server to GET /info/regesstepp/index.html
- Display the resulting page on the screen

More Advanced URLs



Anchor

Jumps to a given section of a page

```
www.textpad.com/download/index.html#downloads
```



Port

For servers running on a port other than the default 80

```
www.bup.edu.bd:8080/secret/marks.txt
```



Query String

A set of parameters passed to a web program

```
www.google.com/search?q=miserable+failure&start=10
```



QUICK CHECK

Let’s Test …

```
http://www.example.org:56789/a/b/c.txt?t=win&s=chess#para5
```

Can you spot each part before we reveal it on the next slide?

Where’s the protocol?	**Where’s the port?**	**Where’s the path?**	**Where’s the query string?**	**Where’s the fragment?**

REVEAL

URLs in More Detail

```
http://www.example.org:56789/a/b/c.txt ?t=win&s=chess #para5
```

Authority of URL

The host (and optional port) that identifies the server: www.example.org:56789

Path of URL / resource

Which resource on that server to fetch: /a/b/c.txt

Query String

Parameters passed to the program: t=win&s=chess

Fragment of HTML

Jumps to a specific section of the page: #para5

ANATOMY OF A MESSAGE

HTTP Request Message

Every HTTP request has the same basic structure:

Start line

Header field(s)

Blank line

Message body (optional)

The start line has three parts, separated by single spaces:

1. Request method
2. Request-URL
3. HTTP version

```
GET /courses/ HTTP/1.1
Host: ce.sharif.edu
Connection: keep-alive
Cache-Control: max-age=0
Accept: text/html,application/xml
User-Agent: Chrome/29.0.1547.66
Accept-Encoding: gzip, deflate
Accept-Language: en-GB,en-US
Cookie: PHPSESSID=91ef778df57d
```

ANATOMY OF A MESSAGE

Request Headers: Name–Value Pairs

Header fields tell the server more about the client and the request — always as name : value pairs.

Host

`ce.sharif.edu`

Which website / server to reach

Connection

`keep-alive`

Keep the connection open for reuse

Accept

`text/html, application/xml`

What content types the client understands

User-Agent

`Chrome/29.0.1547.66`

Which browser & version is asking

Accept-Encoding

`gzip, deflate`

Compression formats the client supports

Cookie

`PHPSESSID=91ef778df57d…`

Session data stored by the browser

REQUEST METHODS

What Can a Request Ask the Server To Do?

GET

Return the resource as the response body.

POST

Send data to be processed by the resource.

HEAD

Return headers only no body. Great for quick checks.

PUT

Store data at this exact Request-URI.

DELETE

Remove the resource at this Request-URI.

OPTION

List the HTTP methods this resource allows.

CONTENT TYPES

Internet Media (MIME) Types

MIME (Multipurpose Internet Mail Extensions) describes what kind of content is being sent also called the *"content-type."*

`text/html`

An HTML document

`image/jpeg`

A JPEG picture

`text/plain`

Human-readable text, no formatting

`video/mp4`

A video file

`application/json`

Structured data for APIs

`application/x-www-form-urlencoded`

Data submitted from a web form

ANATOMY OF A MESSAGE

HTTP Response

The server's reply follows the same shape as a request:

Status line

Header field(s)

Blank line

Message body (optional)

Status codes at a glance:

200 OK **301 Moved** **404 Not Found**

```
HTTP/1.1 200 OK
Date: Sat, 14 Sep 2013 23:09 GMT
Server: Apache/2.2.16 (Debian)
Accept-Ranges: bytes
Content-Encoding: gzip
Content-Length: 1719
Connection: Keep-Alive
Content-Type: text/html

<!DOCTYPE html>
<html>
```

ANATOMY OF A MESSAGE

Common Status Codes

Code	Meaning	What it tells you
200	**OK**	Request processed normally.
301	**Moved Permanently**	The resource has moved to a new URI.
401	**Unauthorized**	A valid password is required.
403	**Forbidden**	The resource is present but read-protected.
404	**Not Found**	No resource matches this Request-URI.
500	**Internal Server Error**	The server hit an internal failure.

HTTP PROTOCOL – STEP BY STEP



CLIENT (Browser)



SERVER (Web Server)

1	User enters URL	https://www.example.com →	Ready to respond
2	Browser sends DNS query	DNS Query → ← Returns IP address	DNS server returns IP address
3	Browser establishes TCP connection	SYN → ← SYN + ACK ACK →	Connection established
4	Browser sends HTTP request	HTTP Request (e.g., GET /) →	Request received
5	Server processes request	Processing... ⚙	Process request
6	Server sends HTTP response	← HTTP Response (e.g., 200 OK)	Response sent
7	Browser renders page and requests additional resources	Additional Requests (CSS, JS, Images, etc.) → ← Additional Responses	Resources served
8	Connection closed	FIN → ← ACK FIN → ← ACK	Connection closed

→ Request (Client → Server) - - -→ Response (Server → Client)

RECAP

The Full Journey: URL Bar to Rendered Page

1



Type a URL

You type or click a link in the browser.

2



DNS lookup

Browser asks DNS to resolve the domain to an IP.

3



TCP connect

A TCP connection opens with that server.

4



HTTP request

Browser sends an HTTP request over that connection.

5



Server responds

Server finds the file and sends back an HTTP response.

6



Page renders

Browser parses the page, fetching images/scripts as needed.

The Web Is Just Client + Server

All web technologies rest on (at least) two kinds of software: **servers** and **clients**. They talk to each other over **HTTP**.



Web Servers

Software that listens for page requests:

- Apache
- Nginx
- Microsoft IIS





Web Clients

Browsers that fetch & display documents:

- Google Chrome
- Mozilla Firefox
- Apple Safari

A WORD OF CAUTION

Security: GET vs. POST

- GET puts everything including sensitive data right in the URL, where it can be seen, bookmarked, and changed. Choose your method carefully.

POST

- More secure by default
- Sends data in the message body
- Handles larger amounts of data
- Best for sensitive or hidden data

GET

- Less secure — visible in the URL
- Sends data in the page address
- Sends comparatively less data
- Great for search / shareable links

HTTP METHOD

GET

Purpose	To retrieve data from the server.
Message headers	Required.
Message body	No message body is sent.

EXAMPLE

From a user database, query to get all users, or a single user by id:

```
http://www.example.com/getUsers
http://www.example.com/getUser?id=1
```



HTTP METHOD

HEAD

Purpose	Same as GET, but the response contains no body only status & header lines.
Message headers	Required.
Message body	No message body is sent.

EXAMPLE

Same endpoints as GET, but only headers come back:

```
http://www.example.com/getUsers
http://www.example.com/getUser?id=1
```



HTTP METHOD

POST

Purpose	To post (create) data on the server.
Message headers	Required.
Message body	XML or JSON formatted data is sent.

EXAMPLE

Create a user named "Alice" in the user database:

```
POST http://www.example.com/createUser
{ "name":"Alice", "age":"25", "department":"ICT", "id":"25" }
```

Note: no data is sent in the URL itself.

HTTP METHOD

PUT

Purpose	To update every occurrence of a specific record on the server.
Message headers	Required.
Message body	Updated XML or JSON formatted data is sent.

EXAMPLE

Update the user named "Alice" in the user database:

```
PUT http://www.example.com/updateUser/25
{ "name":"Alice", "age":27, "department":"ICT" }
```

Note: if the specified data isn't present, PUT can also create it.



HTTP METHOD

DELETE

Purpose	To delete every occurrence of a specific record on the server.
Message headers	Required.
Message body	Can contain a body, but it’s generally useless.

EXAMPLE

Delete the user named "Alice" from the user database:

```
DELETE http://www.example.com/deleteUser?name=Alice
```

Thank You!